

Optimizers, LR Schedules, BatchNorm, Dropout, Early Stopping

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



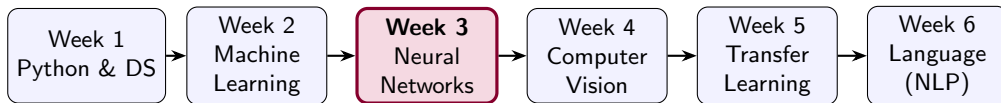
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 17, Lecture

5 October 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. Optimizers
4. Learning-Rate Schedules
5. Batch Normalization
6. Dropout, Weight Decay, Early Stopping
7. Olympiad Practice
8. Limitations and What Comes Next
9. Summary
10. References



Today: Day 17 of 42

- ▶ **Saturday:** the network and backprop. **Yesterday:** the loop as it is really written: the loss, mini-batches, a validation pass, the curve as the dashboard.
- ▶ **Today:** the **dials** of that loop. How each step is taken (optimizers), how big the steps are over time (schedules), how the numbers inside the network are kept tame (BatchNorm), and how memorising is stopped (dropout, weight decay, early stopping).
- ▶ **Then:** three self-study days to go deeper, and Friday's competition, where these dials are the difference between a working network and a stuck one.

Day 16's recipe: $\text{SGD}(\text{lr}=0.05)$, the same rate from the first epoch to the last, nothing between the layers but ReLU, and overfitting handled by keeping the best epoch after running all 120.

Decision	Day 16	What it cost
the step	$w \leftarrow w - \alpha g$, nothing remembered	zigzags in narrow valleys; one step size
the rate	one number, found by trying $\times 10$ and $\nabla \cdot 10$	too big to settle at the end, too small
inside the network	inputs scaled once, layers left alone	deep networks with saturating units do
against memorising	keep the best epoch, run all 120	the gap stays; 107 wasted epochs

Every number in this deck comes from the same MNIST runs as Day 16, so the gains are measured, not promised.

The problem

- ▶ **Stuck.** Five sigmoid layers trained with plain SGD: after ten epochs the loss is 2.306 against 2.303 for guessing, accuracy 0.112. The network never starts.
- ▶ **Memorising.** On 1,000 images the 784-512-10 network reaches training loss 0.002 while the validation loss climbs from 0.387 to 0.479 and the accuracy stays at 0.89. Keeping the best epoch picked a point on the gap; it did not close it.
- ▶ **One rate, two jobs.** 0.05 learns fast then bounces around the bottom; 0.001 settles but needs ten times the epochs (Day 16's curves). Neither is right for the whole run.

The pattern

The loop is right and the data is right; what stalls, bounces or memorises is the way the dials are turned. Each failure has a dial of its own, and each dial is a few lines of PyTorch.

In one sentence

Train well by changing four things about the loop, not the loop: remember past slopes (**momentum, RMSProp, Adam**), shrink the rate over time (**schedules**), re-standardise inside the network (**BatchNorm**), and make memorising expensive or impossible (**dropout, weight decay, early stopping**).

- ▶ Every dial has a definition, a worked example with numbers, a measured gain on the same MNIST runs, and its code: one or two lines each.
- ▶ The reading order of the curve from Day 16 stays the decision tool: the curve says which dial to turn.

Where you will meet it

Every IOAI notebook that trains a network sets these dials in the first ten lines: AdamW, a scheduler, dropout, early stopping on the validation metric. Today's practice task retrains Day 10's ECG beats with them.

By the end of this session, you will be able to:

- ▶ **Explain** what momentum, AdaGrad, RMSProp and Adam each change in the update rule, with numbers, and choose between SGD and Adam for a task.
- ▶ **Schedule** the learning rate (step decay, cosine) and say why a rate that shrinks beats a constant one.
- ▶ **Describe** what BatchNorm computes on a batch, why it needs `eval()`, and where it goes in the network.
- ▶ **Apply** dropout and weight decay, and explain how each one fights the overfitting of Day 16.
- ▶ **Implement** early stopping with patience, and report the epoch you kept.
- ▶ **Combine** the dials into one training recipe and read from the curve which dial to turn.

Training Well: **Optimizers**

Definition

The **optimizer** is the object that turns each dial's slope g into a move. Plain gradient descent (Day 08, `torch.optim.SGD`) is $\theta \leftarrow \theta - \eta g$: the slope times the learning rate η , nothing remembered between steps.

- ▶ Two things go wrong for plain SGD: in a long narrow valley it **zigzags** across the walls instead of running along the floor, and every dial gets the **same step size**, whether its slope is huge or tiny.
- ▶ This section fixes the two problems one at a time, then combines the fixes. Each optimizer keeps a **running average** of something about past slopes.

Optimizer	Fixes	Remembers, per dial	PyTorch
momentum	the zigzag	the average slope	<code>SGD(momentum=0.9)</code>
AdaGrad, RMSProp	one step size for all dials	the average squared slope	<code>Adagrad, RMSprop</code>
Adam, AdamW	both	both averages	<code>Adam, AdamW</code>

momentum: the sideways moves cancel,
the forward moves add up



plain SGD: zigzags, slow along the floor

The ravine problem

The loss valley is steep across and shallow along. Plain SGD bounces off the steep walls and makes slow progress along the floor.

The solution

Give the walk **inertia**: a heavy ball keeps its velocity. Sideways pushes that alternate in sign cancel; pushes in a steady direction add up. Day 16's noisy batch slopes are such pushes.

$$v_t = \underbrace{\beta v_{t-1}}_{\text{past slopes}} + \underbrace{(1 - \beta) g_t}_{\text{new slope}}, \quad \theta_{t+1} = \theta_t - \eta v_t$$

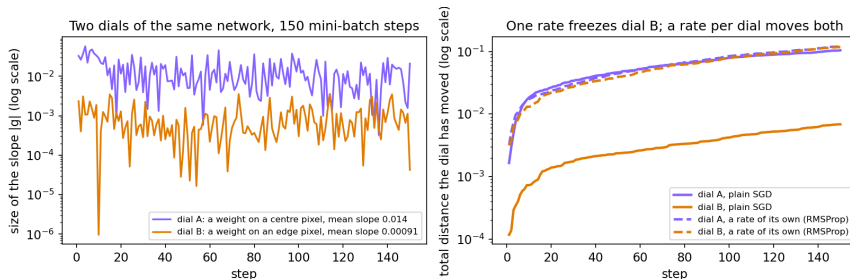
v_t : the **velocity**, a running average of the slopes; β : the momentum coefficient, usually 0.9 (how much of the old velocity survives one step); η : the learning rate.

With numbers

Steady slope $g = 1$ at every step, $\beta = 0.9$: $v = 0.1, 0.19, 0.27, 0.34, \dots \rightarrow 1$: the velocity builds up to the slope and the dial keeps moving at full speed.

Alternating slopes $+1, -1, +1, -1$: $v = 0.1, -0.01, 0.09, -0.02$: the velocity stays near 0 and the dial barely moves. Zigzags cancel; steady directions survive.

PyTorch's `SGD(momentum=0.9)` writes $v_t = \beta v_{t-1} + g_t$ without the $(1 - \beta)$, which only rescales the learning rate: same behaviour, η ten times smaller.



Source: two weights of the first layer of the 784-128-10 MNIST network during 150 mini-batch steps; the moves are what plain SGD (rate 0.05) and a rate-per-dial rule (RMSProp, rate 0.001) would make from the same slopes; `scripts/day17_training_figures.py`.

- **The problem.** Dial A (a centre pixel) has slopes 15 times larger than dial B (an edge pixel). With one rate the move is proportional to the slope: in 150 steps A travels 0.104, B only 0.007. B is as good as frozen.
- **The idea.** Give each dial **its own** rate from its own slope history: large slopes get smaller rates, small slopes larger ones. With RMSProp both dials travel about 0.12.

$$G_t = \underbrace{G_{t-1}}_{\text{past squared slopes}} + \underbrace{g_t^2}_{\text{new}}, \quad \theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{G_t} + \epsilon}}_{\text{this dial's rate}} g_t$$

G_t : the sum of all squared slopes this dial has seen; η : the base rate; $\epsilon \approx 10^{-8}$ keeps the division safe. A large history G_t means a small effective rate.

With numbers

Dial A, slope 2 at every step: $G = 4, 8, 12$, steps $\eta \cdot 2/2, \eta \cdot 2/2.83, \eta \cdot 2/3.46 = \eta \cdot (1.00, 0.71, 0.58)$.

Dial B, slope 0.2 at every step: $G = 0.04, 0.08, 0.12$, steps $\eta \cdot (1.00, 0.71, 0.58)$: the **same** steps as dial A. Each dial's rate is normalised by its own slope size.

The flaw

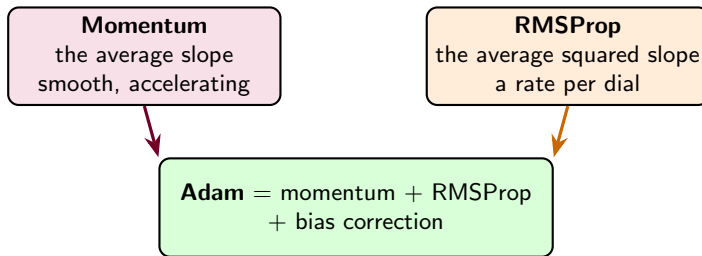
G_t only grows, so every step shrinks: after 10,000 steps with slope 2, $\sqrt{G} = 200$ and the step is $\eta/100$. AdaGrad never forgets, and learning slowly dies.

$$E[g^2]_t = \underbrace{\alpha E[g^2]_{t-1}}_{\text{decayed past}} + \underbrace{(1 - \alpha) g_t^2}_{\text{new squared slope}}, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

- ▶ **The fix:** replace AdaGrad's ever-growing sum by an **exponential moving average** of the squared slopes, decay α usually 0.9 (0.99 in PyTorch): old slopes fade, recent ones count.
- ▶ The effective rate still adapts to each dial, but it no longer dies: it settles at η divided by the dial's *recent* slope size.

With numbers

Slope 2 at every step, $\alpha = 0.9$: $E[g^2] = 0.4, 0.76, 1.08, \dots \rightarrow 4$; steps $\eta \cdot 2 / \sqrt{E} = \eta \cdot (3.16, 2.29, 1.92, \dots) \rightarrow \eta \cdot 1$.
AdaGrad on the same slopes: $\eta \cdot (1.00, 0.71, 0.58, \dots) \rightarrow 0$. RMSPProp stabilises where AdaGrad fades.



- ▶ Keep both running averages: the slope (momentum) gives the **direction**, the squared slope (RMSProp) gives the **step size** of each dial.
- ▶ **Bias correction**: both averages start at 0, so the first ones are too small; Adam divides them by $1 - \beta^t$ to undo that, which matters only in the first steps.
- ▶ Fast, stable, and one learning rate (10^{-3}) that fits almost any network: the most used optimizer in deep learning.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}}_{\text{step size per dial}} \underbrace{\hat{m}_t}_{\text{direction}} \quad \beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$$

With numbers

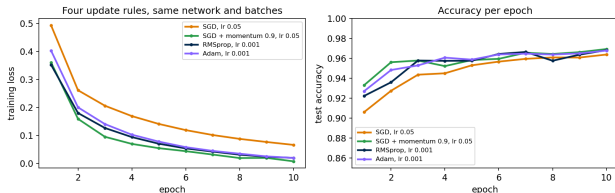
First step, slope $g = 0.2$: $m_1 = 0.02$, $v_1 = 0.00004$; corrected, $\hat{m}_1 = 0.2$, $\hat{v}_1 = 0.04$; the move is $\eta \cdot 0.2/0.2 = \eta$.

First step, slope $g = 20$: $\hat{m}_1 = 20$, $\hat{v}_1 = 400$; the move is $\eta \cdot 20/20 = \eta$ again.

Every dial moves about η per step, whatever the size of its slope; only the sign and the consistency of the slope matter.

That is why $\eta = 0.001$ works on the first try where plain SGD needed Day 16's search; its price is two extra tensors the size of the model.

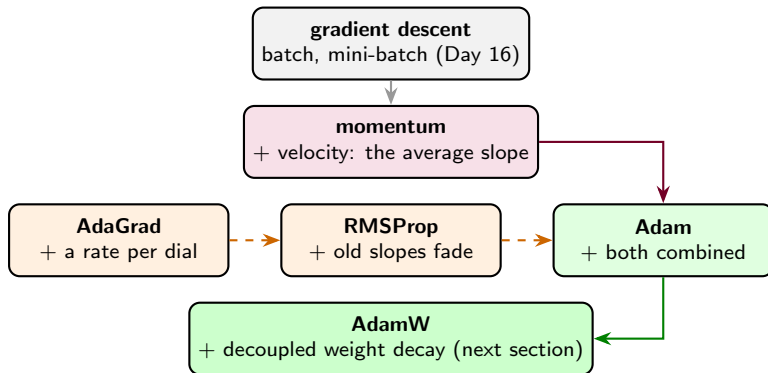
Which Optimizer: the Numbers



Source: 784-128-10 network, 20,000 MNIST images, batches of 64, ten epochs, the same starting dials; `scripts/day17_training_figures.py`.

```
opt = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9)
opt = torch.optim.RMSprop(model.parameters(), lr=1e-3)
opt = torch.optim.Adam(model.parameters(), lr=1e-3) # the usual first try
```

- ▶ Plain SGD reaches 0.95 at epoch 5, momentum at epoch 2, RMSProp and Adam at epoch 3; after ten epochs 0.964, 0.969, 0.968, 0.968 (training loss 0.066, 0.008, 0.020, 0.020).
- ▶ The running averages find the floor that plain SGD zigzags around; RMSProp and Adam get there with no rate search. Try Adam at 0.001 first; SGD with momentum when there is time to tune the rate.



Rule of thumb

Start with AdamW at $\eta = 0.001$ (Adam plus the weight decay of the next section). Switch to SGD with momentum only when there is time to tune the rate and a schedule, where it can end a little higher on images.

Training Well: **Learning-Rate Schedules**

The dilemma of Day 16

A big rate learns fast and then **bounces** around the bottom of the valley without settling; a small rate settles but **crawls** for the first epochs. No single number is right for the whole run.

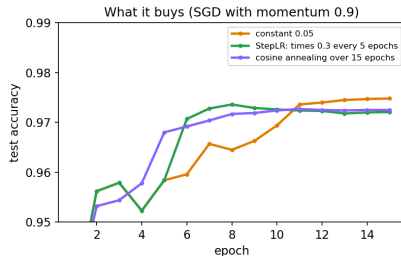
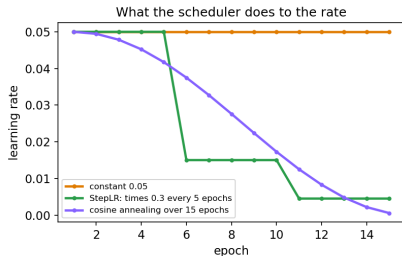
- ▶ A **schedule** changes the learning rate as the epochs pass: big steps while the loss is far from the bottom, small steps to settle into it.
- ▶ **Step decay**: multiply by γ every k epochs. From 0.05, with $\gamma = 0.3$ every 5 epochs: 0.05 for epochs 1 to 5, 0.015 for 6 to 10, 0.0045 for 11 to 15.
- ▶ **Cosine annealing**: a smooth slide from the starting rate to 0 over T epochs, $\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(\pi t / T))$: at $t = 0$ it is 0.05, halfway 0.025, at $t = T$ it is 0.
- ▶ **Reduce on plateau**: cut the rate when the validation loss has not improved for a few epochs: the early-stopping idea applied to the rate.

```
opt = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9)
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=5, gamma=0.3)
# or: torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=15)
# or: torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.3, patience=3)

for epoch in range(15):
    for xb, yb in loader:
        ...
    sched.step()
    print(epoch, opt.param_groups[0]["lr"])
```

- ▶ The scheduler wraps the optimizer and edits its `lr`; `sched.step()` is called once per **epoch**, not per batch (`ReduceLROnPlateau` takes the validation loss as its argument).
- ▶ With Adam a schedule still helps at the end of training, but matters less: Adam already scales each step.

What a Schedule Buys



Source: 784-128-10 network, 20,000 MNIST images, batches of 64, SGD with momentum 0.9 from 0.05, fifteen epochs; `scripts/day17_training_figures.py`.

- ▶ Left: what each scheduler does to the rate. Right: test accuracy after each epoch, from the same starting dials.
- ▶ Fifteen epochs of momentum SGD: constant 0.05 ends at 0.975 with validation loss 0.144; step decay 0.972 with 0.118; cosine 0.973 with 0.126. Here the schedules buy a lower, steadier loss, not a higher accuracy; the accuracy gain shows on long runs and harder data (Week 4).

Training Well: **Batch Normalization**

- ▶ Day 09 and Day 15: inputs are scaled before the first layer so every column has mean 0 and spread 1. Nothing scales what comes *out* of a layer.
- ▶ After a few layers the values of a unit can sit at 20 or at 0.001. A sigmoid at 20 has slope ≈ 0 (Day 15's saturation); a ReLU at -20 is dead. The slopes vanish and the early layers stop learning.
- ▶ Every time the first layer's dials move, the second layer sees inputs with a new mean and spread; it keeps re-learning its scale instead of its job.

Where it breaks

Five layers of 256 sigmoid units on MNIST, plain SGD with learning rate 0.1: after ten epochs the training loss is still 2.306 (guessing is 2.303) and the test accuracy 0.112. The same network with BatchNorm before each sigmoid: loss 0.126 and accuracy 0.939 after ten epochs. Nothing else changed.

Definition

Batch normalization standardises each unit's values across the current mini-batch, then rescales them with two learned dials per unit: $\hat{z} = (z - \mu_B)/\sigma_B$, output $\gamma \hat{z} + \beta$.

One unit, a batch of four

Values $z = (2, 4, 6, 8)$: mean $\mu_B = 5$, spread $\sigma_B = \sqrt{5} = 2.236$.

Standardised: $(-1.342, -0.447, 0.447, 1.342)$. With $\gamma = 2$, $\beta = 1$:
 $(-1.68, 0.11, 1.89, 3.68)$.

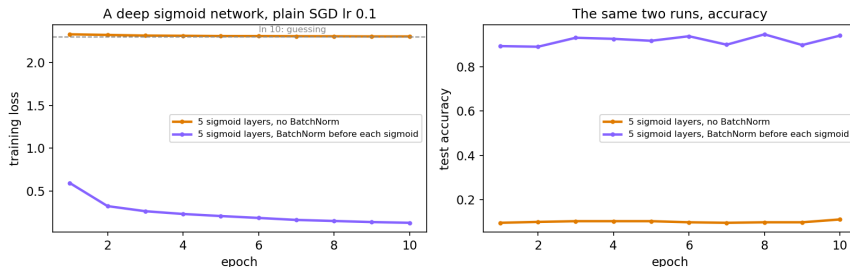
γ and β start at 1 and 0 and are trained like weights; the network can undo the normalisation if that is best, but it starts from tame numbers.

- ▶ Placed after `nn.Linear` and before the activation; `nn.BatchNorm1d(256)` for a layer of 256 units.
- ▶ It also lets you use a bigger learning rate and acts as a light regulariser: each sample's value depends on which other samples share its batch.

```
model = nn.Sequential(  
    nn.Linear(784, 256),  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Linear(256, 256),  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Linear(256, 10))  
  
model.train()    # batch mean and spread  
...  
model.eval()      # running averages  
with torch.no_grad():  
    out = model(X_val)
```

- ▶ In training each batch is standardised with *its own* mean and spread, and the layer keeps a running average of both.
- ▶ At prediction time there may be one sample, so nothing to average: `eval()` switches to the running averages.
- ▶ Forgetting `eval()` is Day 16's warning made concrete: predictions one row at a time become garbage, with no error message.

BatchNorm: the Numbers



Source: five sigmoid layers of 256 units on 20,000 MNIST images, plain SGD with learning rate 0.1, batches of 64, ten epochs; `scripts/day17_training_figures.py`.

- ▶ Without BatchNorm the loss never leaves ln 10: the sigmoids saturate layer by layer and no slope reaches the first layers. With it the loss is 0.59 after one epoch and 0.13 after ten.
- ▶ The BatchNorm accuracy curve wobbles between 0.89 and 0.95 because 0.1 is a large rate for plain SGD; momentum or a schedule smooths it. BatchNorm made the deep network trainable; the other dials make it tidy.

Training Well: **Dropout, Weight Decay, Early Stopping**

- ▶ Day 16's dashboard: 1,000 images, a 784-512-10 network (407,050 dials for 1,000 rows), training loss to 0.002, validation loss turning up at epoch 13.
- ▶ Day 09's cures for a line were a penalty on big weights (Ridge) and choosing the dial on validation rows. Networks keep both and add one of their own.

Cure	What it does	Day 09 relative
dropout	switches off random units during training	none: a network-only trick
weight decay	shrinks every weight a little at each step	Ridge's penalty $\alpha \sum w^2$
early stopping	keeps the best validation epoch, stops after patience	choosing the degree on validation
more data	the only cure that raises the ceiling	the same

Each one is a few lines. The numbers frame at the end of the section shows what each buys on the 1,000-image run.

Definition

Dropout sets each unit of a layer to 0 with probability p , a fresh random choice per batch, and scales the survivors by $1/(1 - p)$. At prediction time nothing is dropped.

With numbers

Units (0.8, 0.6, 0.4, 0.2), $p = 0.5$, mask (1, 0, 1, 0): output (1.6, 0, 0.8, 0). The survivors are doubled, so the sum stays about 2.0 on average: the next layer sees one scale in training and in prediction.

```
# the holes come after the ReLU
model = nn.Sequential(
    nn.Linear(784, 512), nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(512, 10))
model.train()      # holes on
model.eval()       # holes off
```

- ▶ No unit can rely on a neighbour that may vanish, so evidence is spread over many units instead of memorised by a few.
- ▶ Each batch trains a different thinned network and prediction averages them: Day 10's forest inside one network. $p = 0.2$ to 0.5 on hidden layers, never on the output.

$$w \leftarrow w - \alpha (g + \lambda w) \quad (\text{SGD}) \qquad w \leftarrow w - \alpha \lambda w - \text{Adam's step} \quad (\text{AdamW})$$

λ : the decay, a dial you set (10^{-4} to 10^{-2}). Each step shrinks every weight by the fraction $\alpha\lambda$ before the slope moves it.

With numbers

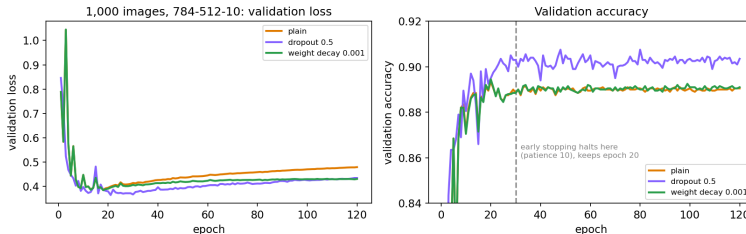
AdamW, $\alpha = 0.001$, $\lambda = 0.01$: each step multiplies every weight by $1 - 10^{-5}$; after 10,000 steps a weight nobody pushes is at 0.905 of its start. A weight the data keeps pulling stays large.

```
params = model.parameters()
opt = torch.optim.SGD(params, lr=0.05, weight_decay=1e-3)
opt = torch.optim.AdamW(params, lr=1e-3, weight_decay=0.01)
```

- For SGD this is Day 09's Ridge gradient added to the slope. With Adam use AdamW; plain Adam(weight_decay=...) decays the wrong amount.

```
best, best_epoch, patience, state = 0.0, 0, 10, None
for epoch in range(1, 200):
    ...                                # train, validate -> val_acc
    if val_acc > best:
        best, best_epoch = val_acc, epoch
        state = copy.deepcopy(model.state_dict())
    elif epoch - best_epoch >= patience: # 10 epochs without a gain
        break
model.load_state_dict(state)
print(f"kept epoch {best_epoch}, validation accuracy {best:.3f}")
```

- ▶ Day 16 kept the best epoch and still ran all 120; this stops patience epochs after the best one, so the epoch count is no longer a dial you guess. patience 5 to 20: too small stops on a noisy dip, too large wastes time.
- ▶ Watch the **metric the task is scored by** (Day 16). The number you report is already the validation score of the kept epoch.



Source: Day 16's run, 1,000 MNIST training images, 784-512-10, batches of 32, SGD with learning rate 0.05, 120 epochs, validation on 2,000 held-out images; `scripts/day17_training_figures.py`.

- Plain: best 0.893 at epoch 20, 0.891 at 120, validation loss up from 0.387 to 0.479. Early stopping (patience 10) halts at epoch 30 and keeps epoch 20: the same result in a quarter of the time.
- Dropout 0.5: best 0.908, still 0.904 at 120, loss 0.365: 1.5 points from the holes. Weight decay 0.001: 0.895, the loss drifting less (0.431). Both together, plus more data, is the recipe.

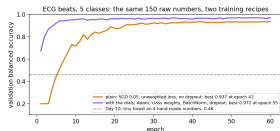
Training Well: **Olympiad Practice**

Task (Day 10, revisited)

2,000 training heartbeats of 150 numbers each, five classes (normal and four disorders), 1,400 of them normal; 1,500 validation beats. Score: **balanced accuracy**, the mean of the five per-class recalls. Day 10 allowed only four hand-made numbers and a ten-tree forest, which reached 0.46. Today the rules are lifted: train a network on the 150 raw numbers with the loop of Day 16 and the dials of today.

- ▶ **Question 1:** 70% of the beats are normal and the metric is balanced accuracy. Which loss, and with what argument?
- ▶ **Question 2:** the first run uses plain SGD, no dropout, no BatchNorm. Which dials do you add, in which order, and which number tells you each one helped?
- ▶ **Question 3:** which epoch's dials do you submit, and how is that decided without touching the hidden test beats?

Adapted from II Olimpiada AI (Poland, 2025), stage 1, task 3 "Wykrywanie zaburzeń sygnału EKG"; data and statement in the OlimpiadaAI GitHub repository.



Source: the task's own training and validation beats, standardised per number; scripts/day17_training_figures.py.

```
w = torch.tensor(n / (5 * counts))
loss_fn = nn.CrossEntropyLoss(weight=w)
model = nn.Sequential(
    nn.Linear(150, 128),
    nn.BatchNorm1d(128), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 64),
    nn.BatchNorm1d(64), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(64, 5))
opt = torch.optim.Adam(model.parameters(),
                        lr=1e-3)
```

1. `CrossEntropyLoss(weight=w)`, w the inverse class shares (Day 16): a missed rare disorder costs as much as a missed normal beat, which is what balanced accuracy measures.
2. Adam at 0.001 so the start does not stall (plain SGD sits at 0.20, the always-normal score, for three epochs), then BatchNorm and dropout 0.3; judge each dial by the validation balanced accuracy, one at a time.
3. The epoch with the best *validation* balanced accuracy, kept with `state_dict()` and early stopping; the hidden test beats are never touched.

Plain network, raw numbers: 0.937 (epoch 42).
With the dials: 0.972 (epoch 55). Day 10's forest: 0.46. The raw signal plus a network is the big step; the dials add 3.5 points.

Training Well: **Limitations & What Comes Next**

- ▶ **Dials fix training, not data.** Dropout and weight decay shrink the gap on 1,000 images; they do not reach what 20,000 images give. When the curve says overfitting, more rows beat every dial.
- ▶ **Adam is not always better.** It wins early and on messy gradients; well-tuned SGD with momentum often ends higher on images and is the default of Week 4's recipes. Try both when the task is close.
- ▶ **BatchNorm depends on the batch.** Tiny batches (below about 16) give noisy means and spreads, and `eval()` must be called or the running averages are not used.
- ▶ **Too many dials to grid-search.** Six dials with four values each is 4,096 runs. The habit: defaults first (Adam 10^{-3} , batch 64, dropout 0.2, early stopping), then change one dial at a time and watch the curve.

The common mistake

Turning three dials at once and crediting whichever one you believe in. Change one, keep the seed fixed, compare the validation curve, then the next. A dial that moves the metric by less than the spread between seeds (Day 09's cross-validation lesson) did nothing.

- ▶ **Days 18 to 20, self-study:** activation functions in depth; tensors, datasets and DataLoaders in depth; then the Olympiad habits for training networks: curves, dials, and the epoch you submit.
- ▶ **Friday:** the Week 3 competition. Everything in it is a network trained with the loop of Day 16 and the dials of today.
- ▶ **Week 4:** the same recipe with convolutional layers, data augmentation inside `--getitem--`, and pretrained networks.

Training Well: **Summary**

- ▶ **Optimizers:** momentum averages the slope and rolls through ravines; AdaGrad and RMSProp give each dial a rate from its own slope history; Adam combines both, so one learning rate (10^{-3}) fits almost every network.
- ▶ **Schedules:** a rate that shrinks (step decay, cosine) takes big steps early and small ones late; `scheduler.step()` once per epoch.
- ▶ **BatchNorm:** each unit is normalised over the batch, then rescaled by two learned dials; deep networks train that could not before; `eval()` switches to the running averages.
- ▶ **Dropout and weight decay:** random holes in training, a penalty on big weights (Day 09's Ridge): both shrink the gap between the training and validation curves.
- ▶ **Early stopping:** watch the validation metric, keep the best epoch, stop after patience epochs without improvement.
- ▶ **The recipe:** defaults first, one dial at a time, the curve decides.

Training Well: **References & Resources**

- [1] Kingma, D. and Ba, J. (2015). Adam: a method for stochastic optimization. *ICLR*. The optimizer of the Adam frames.
- [2] Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. *ICLR*. AdamW, the weight-decay frame.
- [3] Ioffe, S. and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. *ICML*.
- [4] Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. *JMLR* 15.
- [5] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* 32. `torch.optim`, `torch.optim.lr_scheduler`, `nn.BatchNorm1d`, `nn.Dropout`. <https://pytorch.org/docs>
- [6] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*, via OpenML (`mnist_784`); the Day 15 split, reused for every curve of this deck.
- [7] Olimpiada AI (2025). *II Olimpiada Sztucznej Inteligencji, etap 1, zadanie 3: Wykrywanie zaburzeń sygnału EKG*. Day 10's practice task, revisited with a network. https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1_etap

- [8] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, chapters 7 and 8: regularization and optimization.
- [9] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapter 11: faster optimizers, batch normalization, dropout, learning-rate scheduling.
- [10] All plots: `slides-builder/scripts/day17_training_figures.py` (PyTorch runs on real data; it also prints every number quoted in this deck).